CLUSTER ANALYSIS • VISUAL LEARNING GUIDE

Divisive

Hierarchical Clustering

From clustering families and distance measures to a complete worked example.



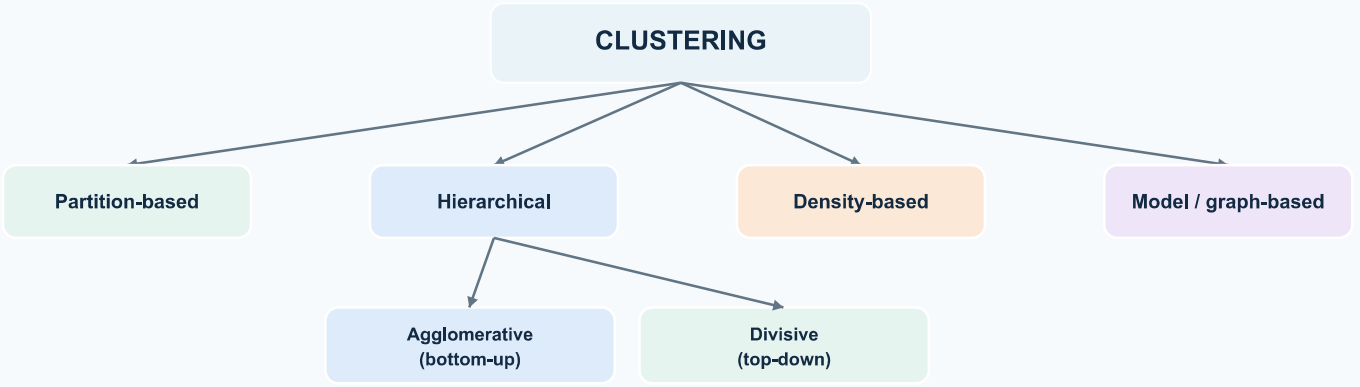
CONCEPTS FLOWCHART WORKED EXAMPLE

Includes the 12-point dataset and splitting rule from your HTML file.

01 / FOUNDATIONS

Clustering methods: the big picture

Different families answer “how should groups be formed?” in different ways.



Examples
K-means • K-medoids

Examples
DBSCAN • HDBSCAN

Examples
Mixture models • spectral

What is divisive hierarchical clustering?

A top-down method that creates nested groups by splitting clusters.

Start at the root
All observations begin together in one cluster. This is the top of the hierarchy.

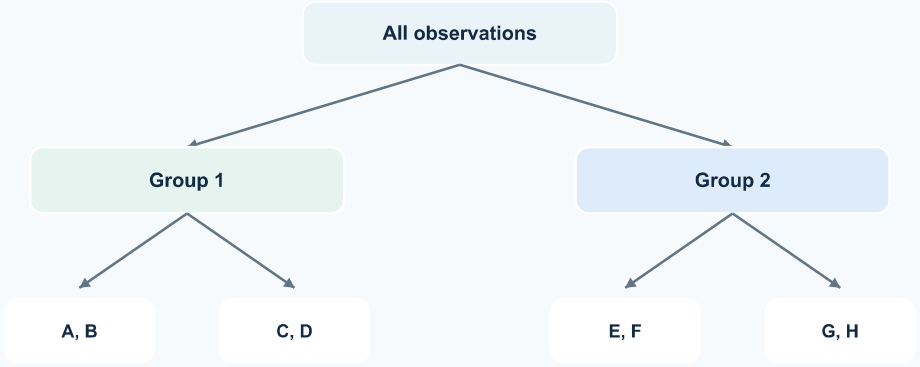
Split one cluster
A rule chooses a cluster and divides it into child clusters.

Stop at the desired level
Stop at singletons, a target number of clusters, or a chosen dissimilarity threshold.

Repeat the process
Each child can be split again, producing progressively finer groups.

How to read a divisive dendrogram

Branches show which groups split from which; leaves are observations.



Read from top to bottom
A parent cluster divides into its children.

Choose a cut level
A horizontal cut gives a flat clustering.

02 / DIVISIVE CLUSTERING

Common divisive splitting strategies

"Divisive" describes a family of procedures, not just one algorithm.

DIANA

Classic divisive analysis: iteratively forms a splinter group using dissimilarity-based rules.

Farthest-seed split

Choose two far-apart observations as seeds; assign remaining points to the nearer seed.

Optimization-based split

Search for a division that improves an objective, often balancing within-cluster cohesion.

Important distinction for this presentation

The worked example follows the farthest-pair + nearest-seed rule in your HTML. It is a teaching algorithm, not the classical DIANA splinter-group procedure.

03 / DISTANCE

Distance: the rule behind the split

The algorithm needs a consistent way to measure how far apart points are.

Euclidean distance

Straight-line distance for numeric coordinates.

Manhattan distance

Sum of absolute coordinate differences; grid-like movement.

Other choices

Cosine, Gower, or domain-specific dissimilarities may fit other data.

For the HTML example: Euclidean distance

$$d(P,Q) = \sqrt{[(xP - xQ)^2 + (yP - yQ)^2]}$$

03 / DISTANCE

Calculate one distance by hand

Example using points A and J from the supplied dataset.

Coordinates

A = (1.1, 8.4)
J = (8.4, 5.8)

Substitute into Euclidean distance

$d(A,J) = \sqrt{[(8.4 - 1.1)^2 + (5.8 - 8.4)^2]}$
$= \sqrt{[7.3^2 + (-2.6)^2]}$
$= \sqrt{(53.29 + 6.76)} \approx 7.749$

Interpretation

A and J are a far-apart pair in the full dataset, so they become the two seeds for the first split.

04 / PROCEDURE

The splitting algorithm used in the HTML

The following rules define the exact worked example.

1. **Start**
   Put every observation into one cluster.
2. **Choose cluster**
   Select the current cluster with the largest diameter.
3. **Choose seeds**
   Find the farthest pair of points inside that cluster.
4. **Assign points**
   Send each remaining point to the nearer seed; ties go to seed A.
5. **Replace & repeat**
   Replace the parent by its two children and continue splitting.

Worked example: the 12 points

These are the exact coordinates from your HTML teaching animation.

Point	x	y
A	1.1	8.4
B	1.8	7.7
C	2.5	8.5
D	2.2	6.9
E	3.0	7.4
F	7.0	8.2
G	7.8	7.6
H	8.6	8.4
I	7.3	6.6
J	8.4	5.8
K	6.6	5.9
L	7.5	5.2

Distance rule

Euclidean distance between each pair of 2D points.

Split rule

Largest-diameter cluster → farthest pair as seeds → assign each point to nearest seed.

Step-by-step split 1: ABCDEFGHIJKL

Groups: ABCDE | JFGHIKL

SPLIT 1 OF 11

Cluster selected: ABCDEFGHIJKL

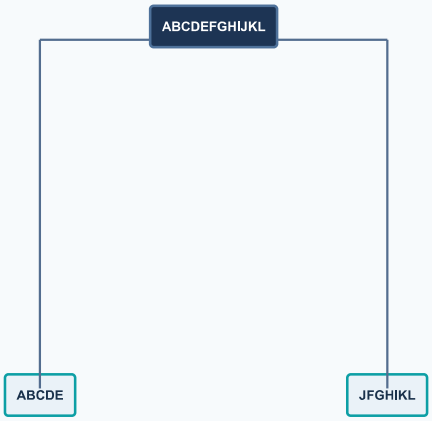
Farthest seeds: A and J

Diameter / seed distance = 7.749

Point	d(to A)	d(to J)	Assigned
B	0.990	6.868	A
C	1.404	6.488	A
D	1.860	6.297	A
E	2.147	5.632	A
F	5.903	2.778	J
G	6.748	1.897	J
H	7.500	2.608	J
I	6.456	1.360	J

Result: ABCDE + JFGHIKL

DENDROGRAM



GROUPS AFTER THIS SPLIT

ABCDE | JFGHIKL

Step-by-step split 2: JFGHIKL

Groups: ABCDE | HFG | LJIK

SPLIT 2 OF 11

Cluster selected: JFGHIKL

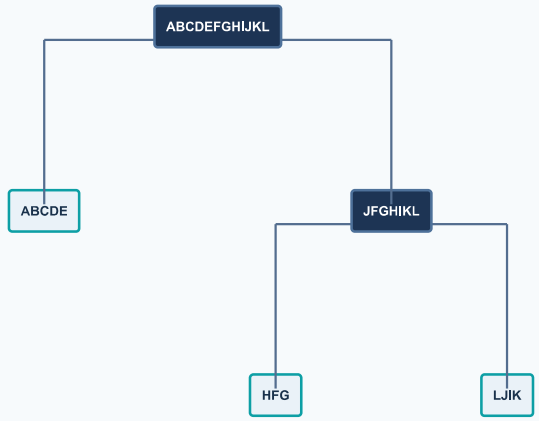
Farthest seeds: H and L

Diameter / seed distance = 3.384

Point	d(to H)	d(to L)	Assigned
J	2.608	1.082	L
F	1.612	3.041	H
G	1.131	2.419	H
I	2.220	1.414	L
K	3.202	1.140	L

Result: HFG + LJIK

DENDROGRAM



GROUPS AFTER THIS SPLIT

ABCDE | HFG | LJIK

Step-by-step split 3: ABCDE

Groups: AB | ECD | HFG | LJIK

SPLIT 3 OF 11

Cluster selected: ABCDE

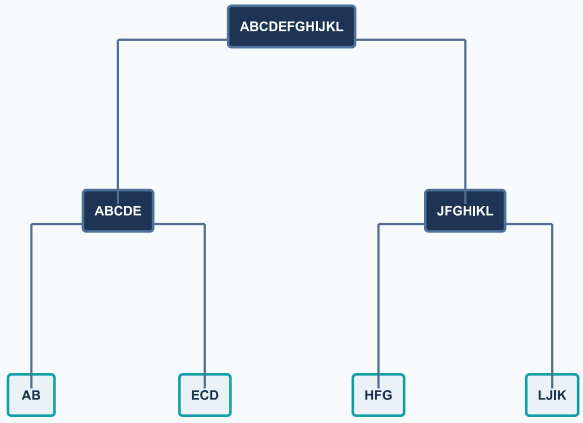
Farthest seeds: A and E

Diameter / seed distance = 2.147

Point	d(to A)	d(to E)	Assigned
B	0.990	1.237	A
C	1.404	1.208	E
D	1.860	0.943	E

Result: AB + ECD

DENDROGRAM



GROUPS AFTER THIS SPLIT

AB | ECD | HFG | LJIK

Step-by-step split 4: LJIK

Groups: AB | ECD | HFG | JL | KI

SPLIT 4 OF 11

Cluster selected: LJIK

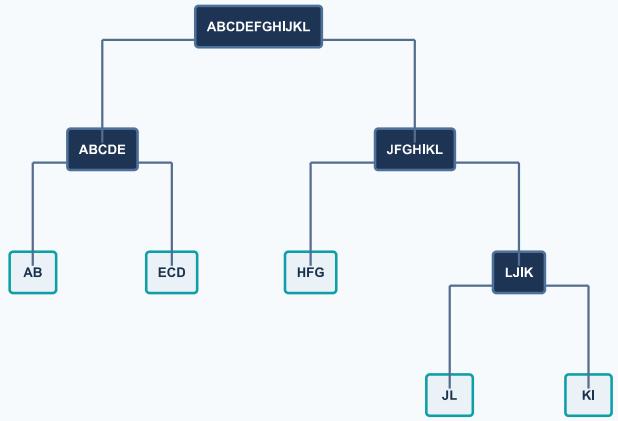
Farthest seeds: J and K

Diameter / seed distance = 1.803

Point	d(to J)	d(to K)	Assigned
L	1.082	1.140	J
I	1.360	0.990	K

Result: JL + KI

DENDROGRAM



GROUPS AFTER THIS SPLIT

AB | ECD | HFG | JL | KI

Step-by-step split 5: ECD

Groups: AB | C | DE | HFG | JL | KI

SPLIT 5 OF 11

Cluster selected: ECD

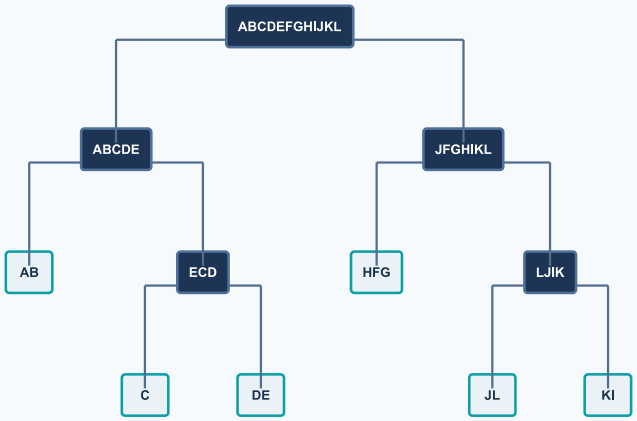
Farthest seeds: C and D

Diameter / seed distance = 1.628

Point	d(to C)	d(to D)	Assigned
E	1.208	0.943	D

Result: C + DE

DENDROGRAM



GROUPS AFTER THIS SPLIT

AB | C | DE | HFG | JL | KI

Step-by-step split 6: HFG

Groups: AB | C | DE | H | FG | JL | KI

SPLIT 6 OF 11

Cluster selected: HFG

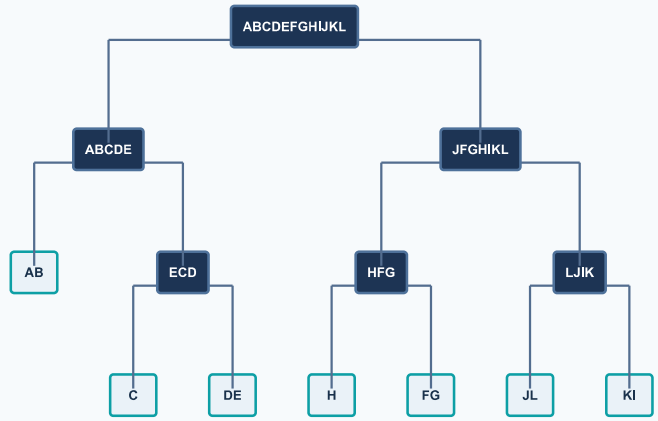
Farthest seeds: H and F

Diameter / seed distance = 1.612

Point	d(to H)	d(to F)	Assigned
G	1.131	1.000	F

Result: H + FG

DENDROGRAM



GROUPS AFTER THIS SPLIT

AB | C | DE | H | FG | JL | KI

Step-by-step split 7: JL

Groups: AB | C | DE | H | FG | J | L | KI

SPLIT 7 OF 11

Cluster selected: JL

Farthest seeds: J and L

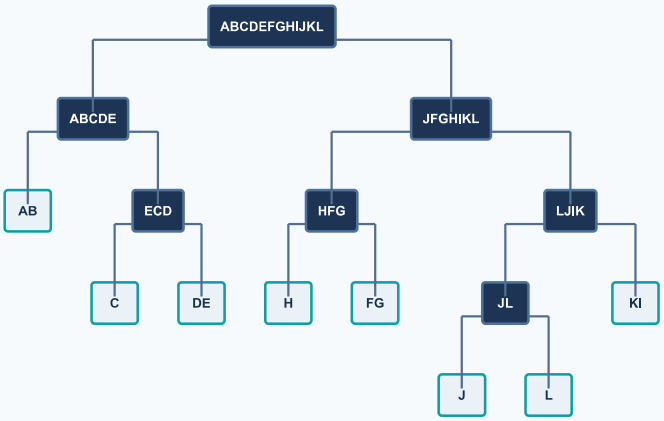
Diameter / seed distance = 1.082

Point	d(to J)	d(to L)	Assigned

Only the two seed points remain.

Result: J + L

DENDROGRAM



GROUPS AFTER THIS SPLIT

AB | C | DE | H | FG | J | L | KI

Step-by-step split 8: FG

Groups: AB | C | DE | H | F | G | J | L | KI

SPLIT 8 OF 11

Cluster selected: FG

Farthest seeds: F and G

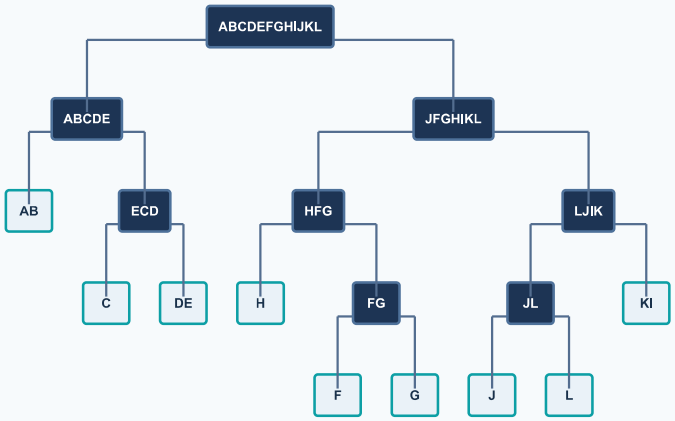
Diameter / seed distance = 1.000

Point d(to F) d(to G) Assigned

Only the two seed points remain.

Result: F + G

DENDROGRAM



GROUPS AFTER THIS SPLIT

AB | C | DE | H | F | G | J | L | KI

Step-by-step split 9: AB

Groups: A | B | C | DE | H | F | G | J | L | KI

SPLIT 9 OF 11

Cluster selected: AB

Farthest seeds: A and B

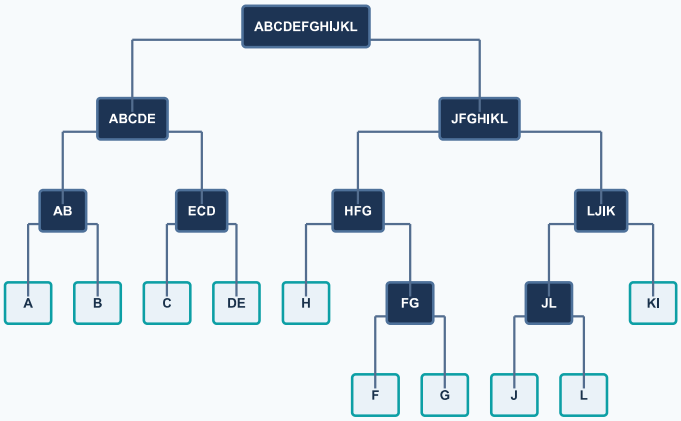
Diameter / seed distance = 0.990

Point d(to A) d(to B) Assigned

Only the two seed points remain.

Result: A + B

DENDROGRAM



GROUPS AFTER THIS SPLIT

A | B | C | DE | H | F | G | J | L | KI

Step-by-step split 10: KI

Groups: A | B | C | DE | H | F | G | J | L | K | I

SPLIT 10 OF 11

Cluster selected: KI

Farthest seeds: K and I

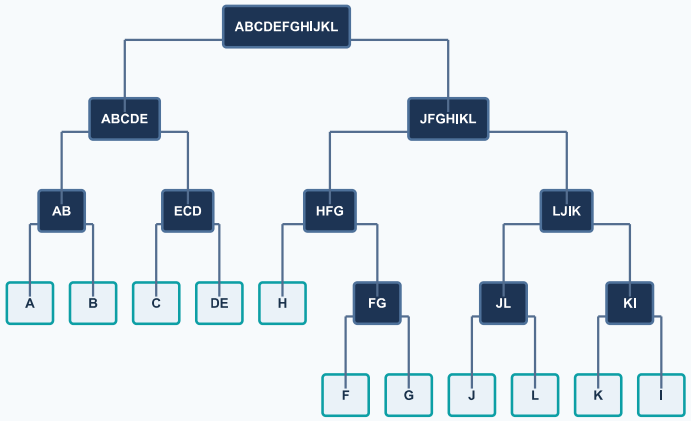
Diameter / seed distance = 0.990

Point	d(to K)	d(to I)	Assigned

Only the two seed points remain.

Result: K + I

DENDROGRAM



GROUPS AFTER THIS SPLIT

A | B | C | DE | H | F | G | J | L | K | I

Step-by-step split 11: DE

Groups: A | B | C | D | E | F | G | H | I | J | K | L

SPLIT 11 OF 11

Cluster selected: DE

Farthest seeds: D and E

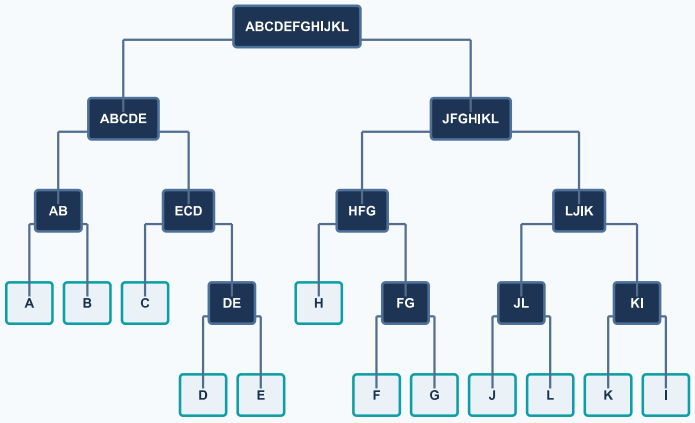
Diameter / seed distance = 0.943

Point	d(to D)	d(to E)	Assigned

Only the two seed points remain.

Result: D + E

DENDROGRAM



GROUPS AFTER THIS SPLIT

A | B | C | D | E | H | F | G | J | L | K | I

07 / PRACTICE

Practice: work through the splits

For each question, use Euclidean distance and the farthest-pair / nearest-seed rule.

1

Question 1 • Two obvious groups

Points: A(1,1), B(1,2), C(2,1), D(7,7), E(8,7). Find the farthest pair, assign the remaining points to the nearer seed, and write the two groups.

2

Question 2 • Calculate before assigning

Points: A(0,0), B(2,0), C(1,1), D(6,0). Start with one cluster. Calculate the pairwise distances, choose the two seeds, and show the split.

3

Question 3 • Continue a hierarchy

A cluster contains P(1,1), Q(2,1), R(1,3), S(7,7). After finding its first split, identify which child has the larger diameter and should be split next.

07 / PRACTICE

Practice: explain your dendrogram

Show your calculations and label every group after each split.

1

Question 4 • Distance and tie-breaking

Points: A(0,0), B(2,0), C(1,0), D(5,0). Choose the farthest pair as seeds, assign each other point to the nearest seed, and explain any tie.

2

Question 5 • Make the next split

Current groups are {A,B,C,D} and {E,F,G}. Their diameters are 5.2 and 3.7. Which group is split next? State the rule and the information you need to split it.

3

Question 6 • Read the result

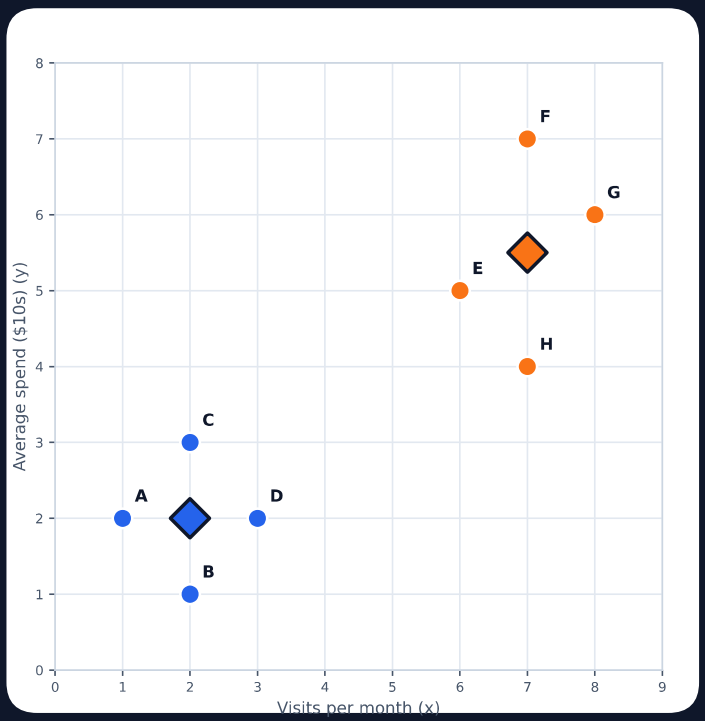
A split produces {A,B,C} and {D,E}. Write the next possible splits until every observation is a singleton, using the farthest-pair rule within the selected cluster.

K-means Clustering

Finding groups in data when nobody has told us the groups

Master's Statistics / Data Science · Cluster Analysis

A step-by-step teaching module with a fully worked example



Learning Objectives & Where We Are

1. Explain what clustering and K-means do, and what K means
2. Use Euclidean distance, centroids and WCSS correctly
3. Run the algorithm by hand, one iteration at a time
4. Choose K with the elbow method; know the pitfalls
5. Compare K-means with hierarchical clustering and K-medoids

Course roadmap

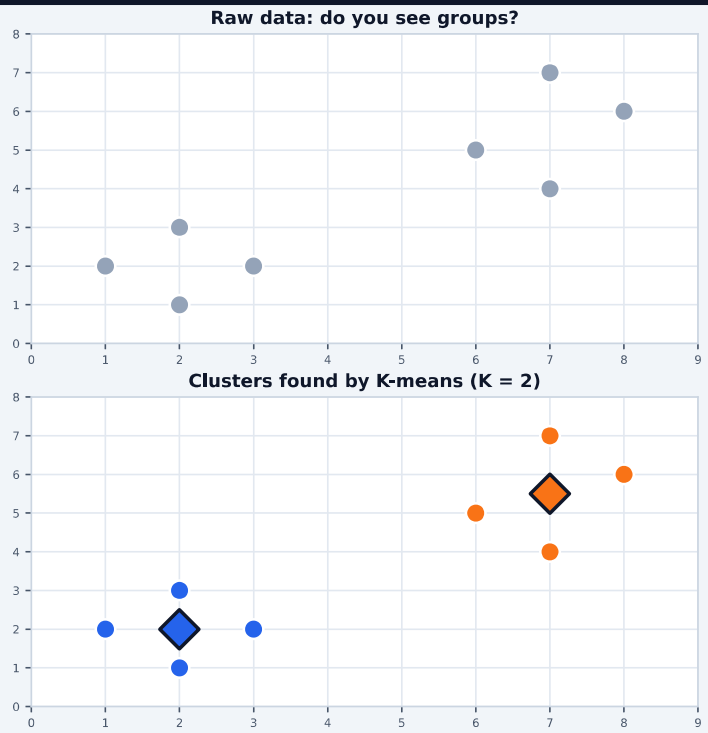
- Preprocessing, variables, train/test
- Supervised: regression, logistic, KNN, Naive Bayes, trees, RF, SVM
- Similarity & proximity measures
- Cluster analysis: types of clustering
- **K-means (today)**
- Hierarchical & dendrograms

Introduction to Clustering

- Clustering groups observations so members of a group are similar to each other and different from other groups.
- It is unsupervised learning: no response variable Y or labels, unlike regression, KNN or trees.
- Customer segmentation · gene-expression groups · topic grouping of documents · colour compression of images

Checkpoint 1

Is predicting house price from floor area an example of clustering? (Answer on the Answers page.)



Essential Prerequisites

Observation

One row of the data: a single customer (a point).

Feature

One column: a measured variable such as visits or spend. Must be numeric for K-means.

Cluster

A group of observations that are close to one another.

Centroid

The mean point of a cluster. It need not be a real observation.

Similarity / dissimilarity

How alike / different two points are. A distance is a dissimilarity.

Euclidean distance

$d = \sqrt{((x_1-c_1)^2 + (x_2-c_2)^2)}$: the straight-line distance.

Checkpoint 2: What is the Euclidean distance between A(1, 2) and B(2, 1)? (Answer on the Answers page.)

Understanding K-means

K-means splits n observations into exactly K clusters. Each point joins the cluster with the nearest centroid.

- K is chosen by you before running the algorithm.
- K = number of clusters = number of centroids.
- Goal: make clusters as compact as possible (small WCSS).

	K-means	Classification (KNN, trees)	Hierarchical
Labels	None	Known	None
K	Fixed in advance	Classes given	Cut the dendrogram
Output	Flat partition	Predicted class	Nested tree
Speed	Fast	—	Slower (n^2 dist.)

Mathematical Foundation

$$d(x, c) = \sqrt{(x_1 - c_1)^2 + (x_2 - c_2)^2}$$

Euclidean distance

How far a point x is from a centroid c: difference in each coordinate, square, add, take the square root.

$$\mu_k = \frac{1}{n_k} \sum_{x \in C_k} x$$

Centroid (cluster mean)

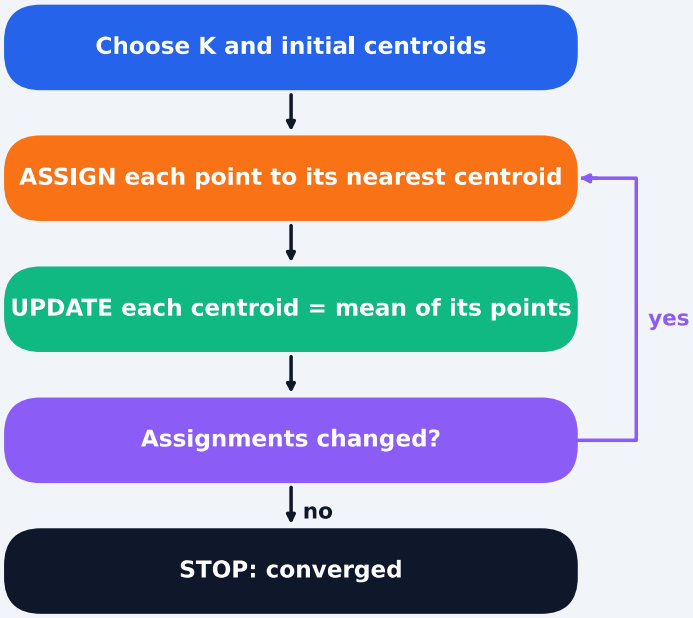
μ_k = centroid of cluster k; n_k = number of points in it; add up the points and divide by n_k, coordinate by coordinate.

$$\text{WCSS} = \sum_{k=1}^{K} \sum_{x \in C_k} \|x - \mu_k\|^2$$

Objective: within-cluster sum of squares

For every cluster, add the squared distances from each point to its own centroid. K-means tries to make this as small as possible.

Algorithm & Flowchart




```
choose K;  initialise μ₁ … μ_K
repeat
   for each x:  label(x) = argmin_k ‖x − μ_k‖
   for each k:  μ_k = mean{ x : label(x) = k }
until labels no longer change
      (or max iterations reached)
```

General algorithm vs. our example

Our example: K = 2, initial centroids = two data points (B and D), Euclidean distance, ties go to the lower cluster number, stop when assignments repeat.

Worked Example: The Data

Scenario

8 customers. x = store visits per month, y = average spend (in $10s). We want K = 2 segments.

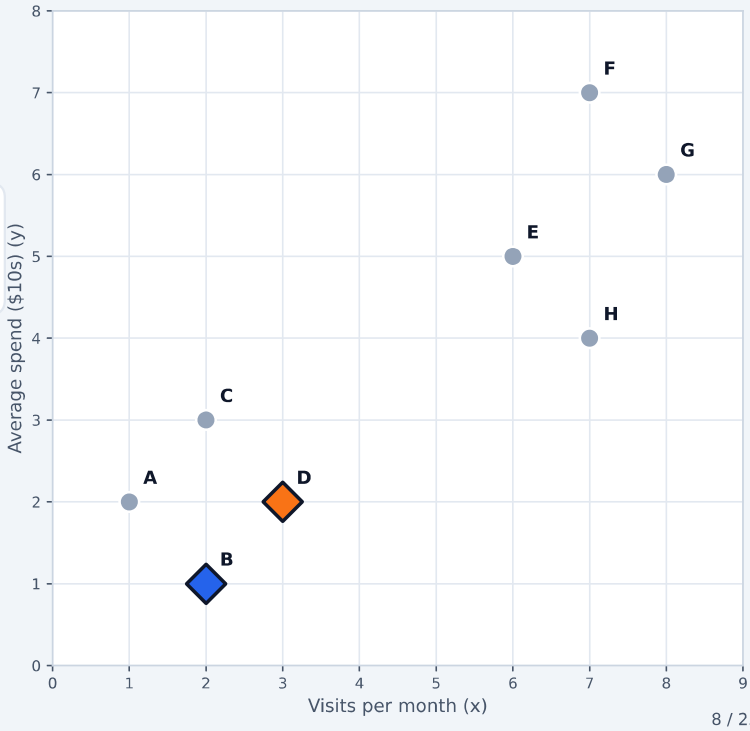
Point	x	y
A	1	2
B	2	1
C	2	3
D	3	2
E	6	5
F	7	7
G	8	6
H	7	4

Start

C1 = B (2,1) C2 = D (3,2)

Rules used

Euclidean distance · ties → C1 · stop when assignments do not change. Different starting points can give different final solutions.



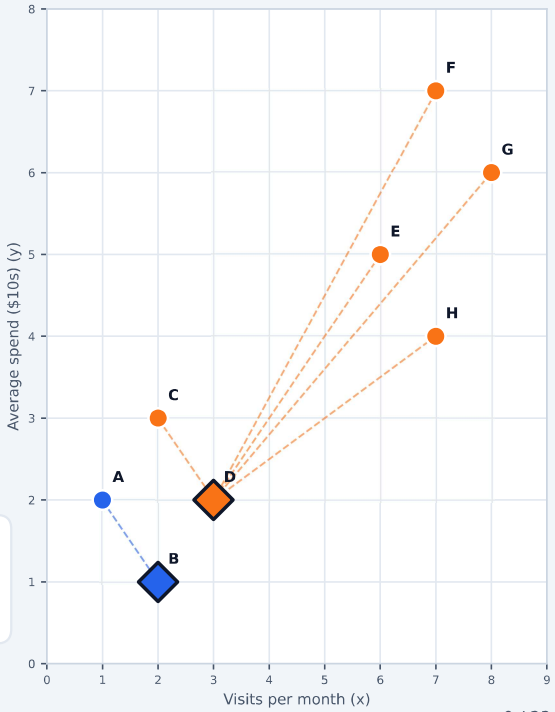
Iteration 1 · Distances → Assignment

Current centroids: C1 = (2, 1) C2 = (3, 2)

Point	d to C1	d to C2	Joins
A (1,2)	**√((1−2)²+(2−1)²) = 1.41**	√((1−3)²+(2−2)²) = 2	C1
B (2,1)	**√((2−2)²+(1−1)²) = 0**	√((2−3)²+(1−2)²) = 1.41	C1
C (2,3)	√((2−2)²+(3−1)²) = 2	**√((2−3)²+(3−2)²) = 1.41**	C2
D (3,2)	√((3−2)²+(2−1)²) = 1.41	**√((3−3)²+(2−2)²) = 0**	C2
E (6,5)	√((6−2)²+(5−1)²) = 5.66	**√((6−3)²+(5−2)²) = 4.24**	C2
F (7,7)	√((7−2)²+(7−1)²) = 7.81	**√((7−3)²+(7−2)²) = 6.4**	C2
G (8,6)	√((8−2)²+(6−1)²) = 7.81	**√((8−3)²+(6−2)²) = 6.4**	C2
H (7,4)	√((7−2)²+(4−1)²) = 5.83	**√((7−3)²+(4−2)²) = 4.47**	C2

Rule

Each point joins the centroid with the smaller distance (highlighted). A tie would go to C1. Initial centroids are the points B and D.



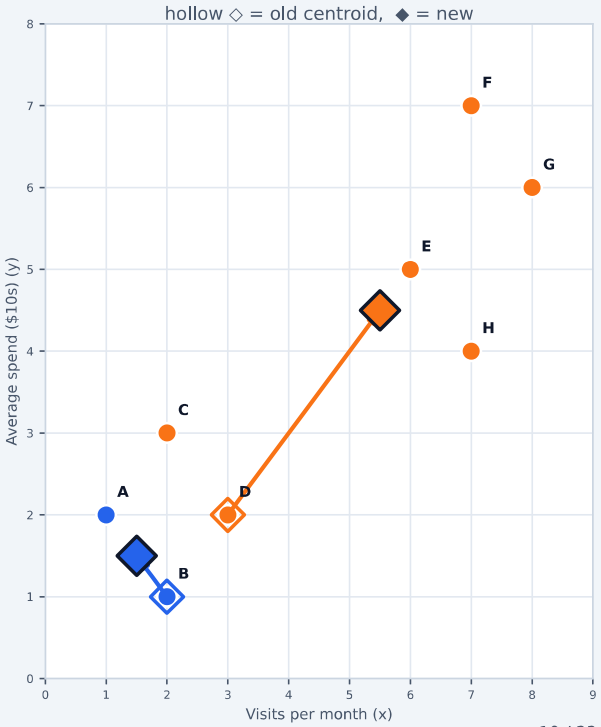
Iteration 1 · Update Centroids

```
C1 = {A, B}
   x: (1+2) / 2 = 1.5
   y: (2+1) / 2 = 1.5
   → new C1 = (1.5, 1.5)
```

```
C2 = {C, D, E, F, G, H}
   x: (2+3+6+7+8+7) / 6 = 5.5
   y: (3+2+5+7+6+4) / 6 = 4.5
   → new C2 = (5.5, 4.5)
```

WCSS with these centroids = 48

Assignments changed (or first pass), so centroids moved and we repeat.



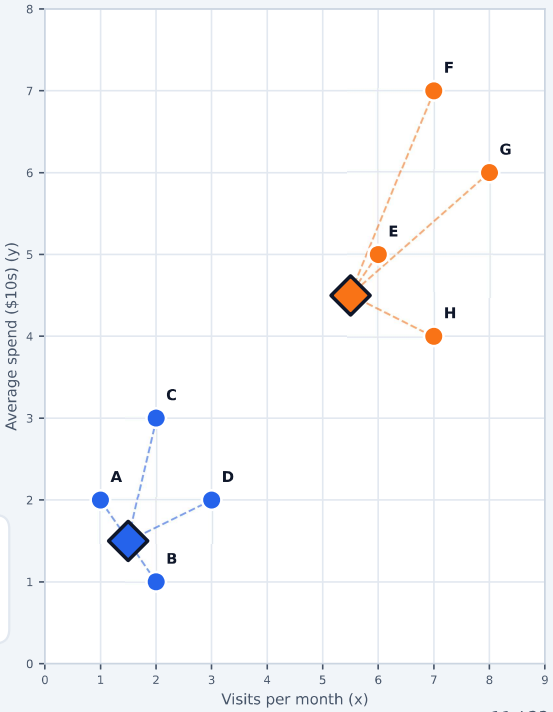
Iteration 2 · Distances → Assignment

Current centroids: C1 = (1.5, 1.5) C2 = (5.5, 4.5)

Point	d to C1	d to C2	Joins
A (1,2)	**√((1−1.5)²+(2−1.5)²) = 0.71**	√((1−5.5)²+(2−4.5)²) = 5.15	C1
B (2,1)	**√((2−1.5)²+(1−1.5)²) = 0.71**	√((2−5.5)²+(1−4.5)²) = 4.95	C1
C (2,3)	**√((2−1.5)²+(3−1.5)²) = 1.58**	√((2−5.5)²+(3−4.5)²) = 3.81	C1
D (3,2)	**√((3−1.5)²+(2−1.5)²) = 1.58**	√((3−5.5)²+(2−4.5)²) = 3.54	C1
E (6,5)	√((6−1.5)²+(5−1.5)²) = 5.7	**√((6−5.5)²+(5−4.5)²) = 0.71**	C2
F (7,7)	√((7−1.5)²+(7−1.5)²) = 7.78	**√((7−5.5)²+(7−4.5)²) = 2.92**	C2
G (8,6)	√((8−1.5)²+(6−1.5)²) = 7.91	**√((8−5.5)²+(6−4.5)²) = 2.92**	C2
H (7,4)	√((7−1.5)²+(4−1.5)²) = 6.04	**√((7−5.5)²+(4−4.5)²) = 1.58**	C2

Rule

Each point joins the centroid with the smaller distance (highlighted). A tie would go to C1.



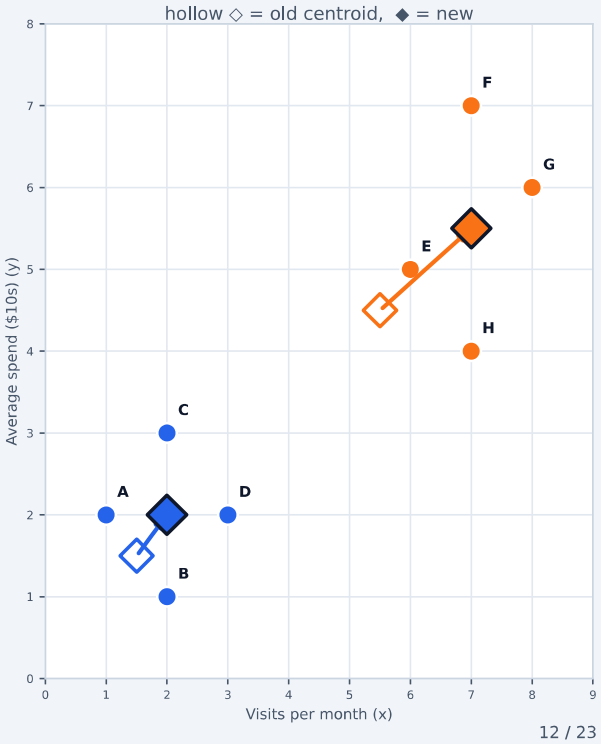
Iteration 2 · Update Centroids

```
C1 = {A, B, C, D}
   x: (1+2+2+3) / 4 = 2
   y: (2+1+3+2) / 4 = 2
   → new C1 = (2, 2)
```

```
C2 = {E, F, G, H}
   x: (6+7+8+7) / 4 = 7
   y: (5+7+6+4) / 4 = 5.5
   → new C2 = (7, 5.5)
```

WCSS with these centroids = 11

Assignments changed (or first pass), so centroids moved and we repeat.



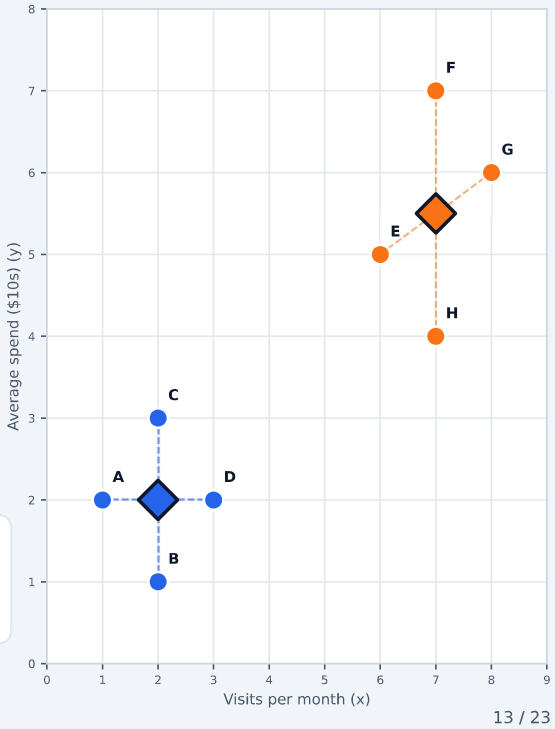
Iteration 3 · Distances → Assignment

Current centroids: C1 = (2, 2) C2 = (7, 5.5)

Point	d to C1	d to C2	Joins
A (1,2)	**$\sqrt{((1-2)^2+(2-2)^2)} = 1$**	$\sqrt{((1-7)^2+(2-5.5)^2)} = 6.95$	C1
B (2,1)	**$\sqrt{((2-2)^2+(1-2)^2)} = 1$**	$\sqrt{((2-7)^2+(1-5.5)^2)} = 6.73$	C1
C (2,3)	**$\sqrt{((2-2)^2+(3-2)^2)} = 1$**	$\sqrt{((2-7)^2+(3-5.5)^2)} = 5.59$	C1
D (3,2)	**$\sqrt{((3-2)^2+(2-2)^2)} = 1$**	$\sqrt{((3-7)^2+(2-5.5)^2)} = 5.32$	C1
E (6,5)	$\sqrt{((6-2)^2+(5-2)^2)} = 5$	**$\sqrt{((6-7)^2+(5-5.5)^2)} = 1.12$**	C2
F (7,7)	$\sqrt{((7-2)^2+(7-2)^2)} = 7.07$	**$\sqrt{((7-7)^2+(7-5.5)^2)} = 1.5$**	C2
G (8,6)	$\sqrt{((8-2)^2+(6-2)^2)} = 7.21$	**$\sqrt{((8-7)^2+(6-5.5)^2)} = 1.12$**	C2
H (7,4)	$\sqrt{((7-2)^2+(4-2)^2)} = 5.39$	**$\sqrt{((7-7)^2+(4-5.5)^2)} = 1.5$**	C2

Rule

Each point joins the centroid with the smaller distance (highlighted). A tie would go to C1.



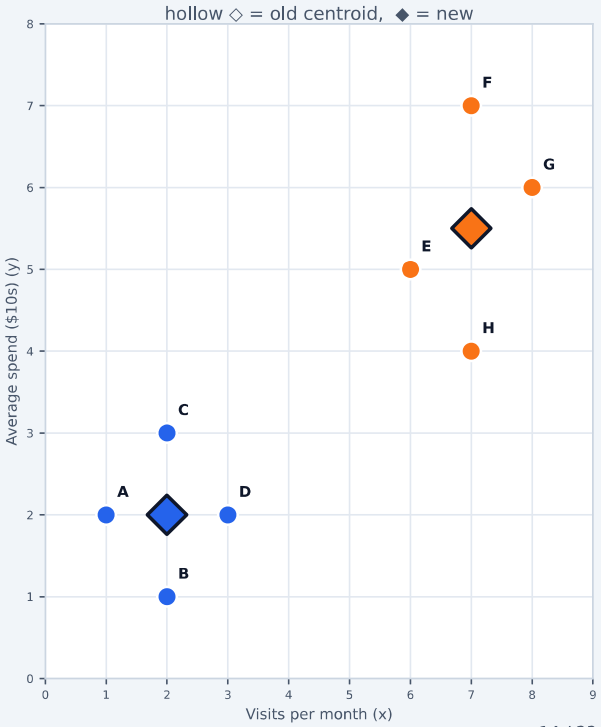
Iteration 3 · Update Centroids

```
C1 = {A, B, C, D}
   x: (1+2+2+3) / 4 = 2
   y: (2+1+3+2) / 4 = 2
   → new C1 = (2, 2)
```

```
C2 = {E, F, G, H}
   x: (6+7+8+7) / 4 = 7
   y: (5+7+6+4) / 4 = 5.5
   → new C2 = (7, 5.5)
```

WCSS with these centroids = 11

Assignments are identical to the previous iteration, so centroids no longer move. CONVERGED: stop.



Interpreting the Final Result

Cluster 1: {A, B, C, D}

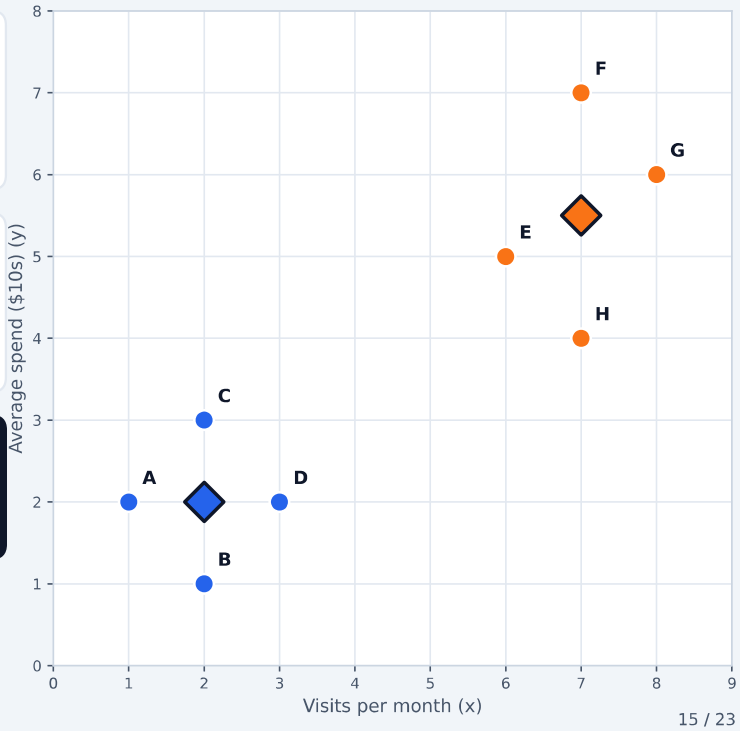
Centroid (2, 2) · WCSS = 4
Interpretation: Occasional, low-spend customers

Cluster 2: {E, F, G, H}

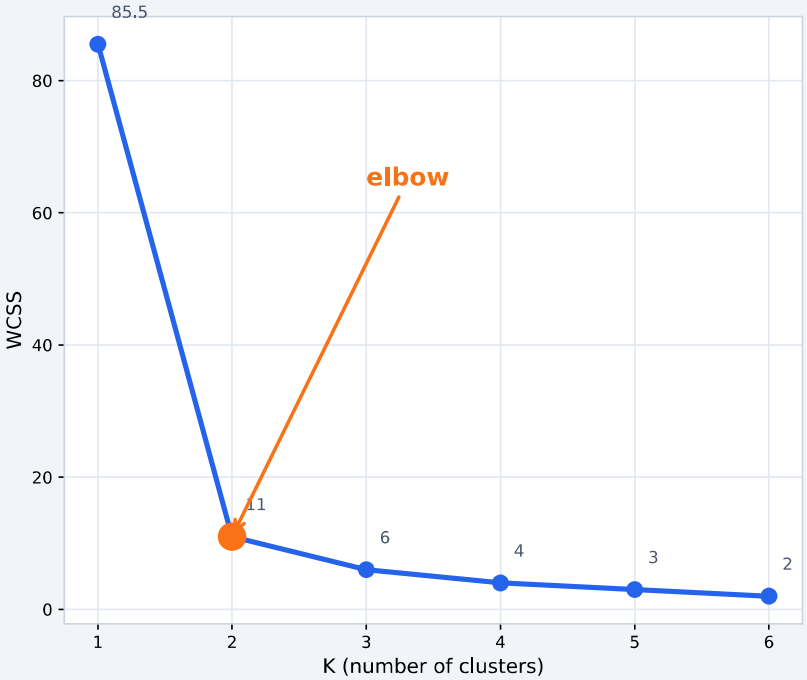
Centroid (7, 5.5) · WCSS = 7
Interpretation: Frequent, high-spend customers

Total WCSS = 11 after 3 iterations

- K-means gives groups only; the business labels are our interpretation.
- Another start might give another answer.



Choosing K: The Elbow Method



- Run K-means for K = 1, 2, 3 … and record WCSS.
- WCSS always decreases as K grows.
- Look for the elbow: where adding clusters stops paying off.
- Here the big drop is K = 1 → 2, then the curve flattens.

Caution

Real data often give a smooth curve with no clear elbow. Combine with silhouette, stability and subject knowledge.

Answers & Explanations

Checkpoint 1

No. House price is a known response, so this is supervised (regression).

Checkpoint 2

$\sqrt{((1-2)^2 + (2-1)^2)} = \sqrt{2} \approx 1.41$

Checkpoint 3

Keep the one with WCSS = 11: both are local optima, lower WCSS is better.

Q1: b

K is the user-chosen number of clusters.

Q2: c

The mean minimises the sum of squared distances within a cluster.

Q3: a

Euclidean distance mixes units; large-range variables would dominate.

Exercise

d to C1 = $\sqrt{((5-2)^2+(4-2)^2)} = \sqrt{13} \approx 3.61$; d to C2 = $\sqrt{((5-7)^2+(4-5.5)^2)} = \sqrt{6.25} = 2.5$. $2.5 < 3.61 \rightarrow$ Cluster 2.

Summary & Take-Home Message

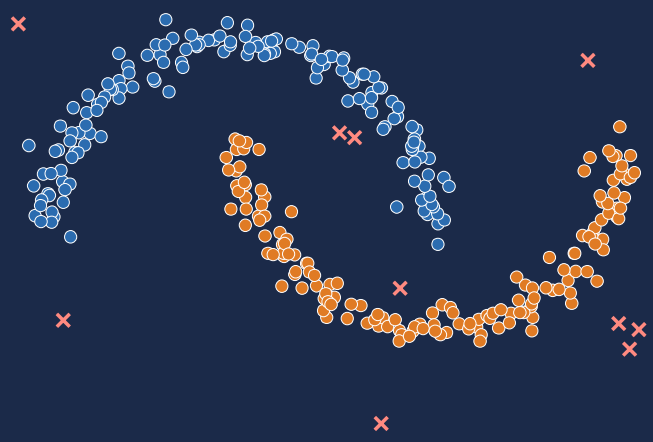
1 Choose K and initial centroids | **2 ASSIGN to the nearest centroid** | **3 UPDATE centroids to cluster means** | **4 REPEAT until assignments stop**

$$\text{minimise} \quad \text{WCSS} = \sum_{k} \sum_{x \in C_k} \|x - \mu_k\|^2$$

- Unsupervised, numeric data, K fixed in advance; always converges, to a local optimum.
- Our example: 3 iterations, final centroids (2,2) and (7,5.5), WCSS = 11.
- Scale features, use several starts, choose K with the elbow plus judgement.
- Next: similarity measures, hierarchical clustering and dendrograms.

Density-Based Clustering (DBSCAN)

Finding clusters of any shape, and noise, by following regions of high density



red × = noise points left unclustered

Course context
Master's-level Statistics / Data Science
Unit: Cluster Analysis | Non-hierarchical clustering
Builds on: distance measures, KNN, K-means

Learning Objectives

By the end of this module you will be able to:

- Explain why density is a different idea of a 'cluster' from compactness
- Define ε, MinPts, core, border and noise points
- Explain density-reachable and density-connected, and how they define a cluster
- Run DBSCAN by hand, one step at a time, on a small dataset
- Choose ε and MinPts with the k-distance plot and understand their effects
- Say when DBSCAN beats K-means and when it struggles

Where DBSCAN fits in the course

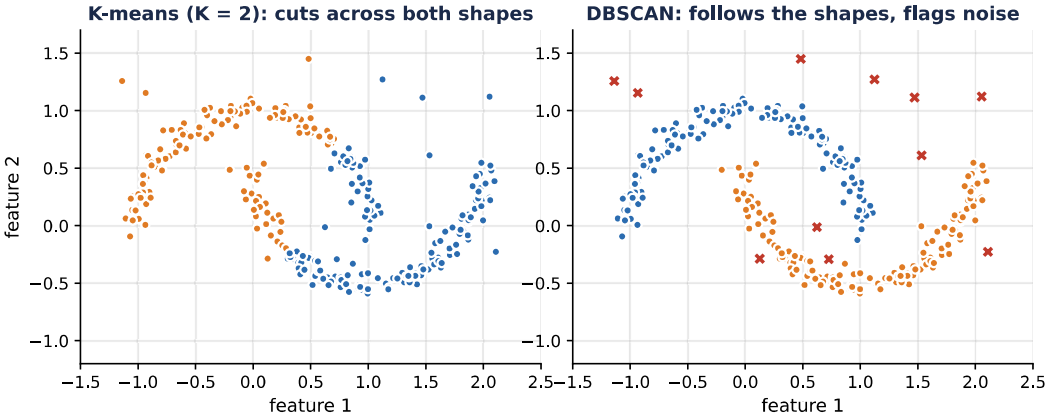
Already studied
- Distances, scaling, preprocessing
- KNN: neighbours within a radius
- K-means: partitioning, centroids

Today: DBSCAN
- Non-hierarchical, density-based
- No K needed; finds noise
- Shape-free clusters

Coming next
- Hierarchical clustering, dendrograms
- Choosing between methods
- OPTICS / HDBSCAN (extensions)

Why Density-Based Clustering?

K-means draws round, compact groups. Real clusters are often curved, and some points belong to no cluster.



The density idea

A cluster is a region where points are packed closely together, separated from other regions by sparse space.

Points in sparse space are noise.

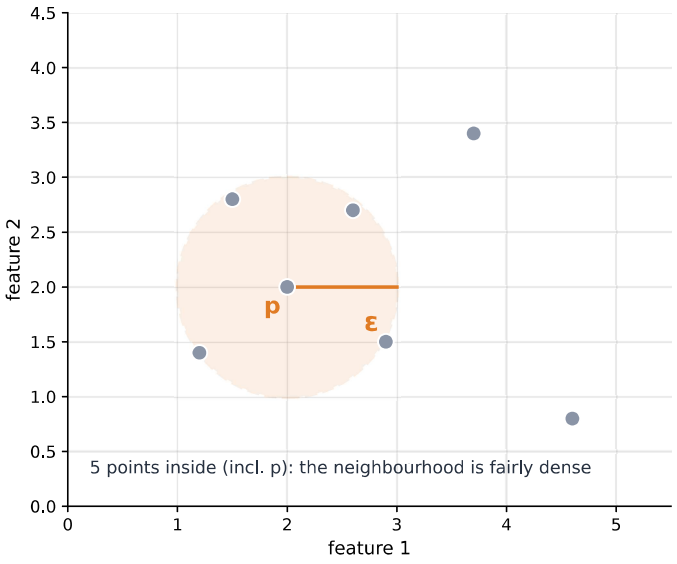
Think of islands in the sea

Dense land = cluster, empty water = gaps, a lone rock = noise. You can walk from any point on an island to any other without leaving the land.

Essential Vocabulary (1/2)

Neighbourhood and density

- Distance: we use the same Euclidean distance as in K-means and KNN.
- ε-neighbourhood of a point p: all points within distance ε of p (a circle in 2-D, a ball in higher dimensions).
- Density around p: how many points fall inside that neighbourhood. Many points = dense, few = sparse.
- Recall K-means: it needed K and centroids. DBSCAN needs no K and no centroids, only a notion of 'close' (ε) and 'crowded' (MinPts).



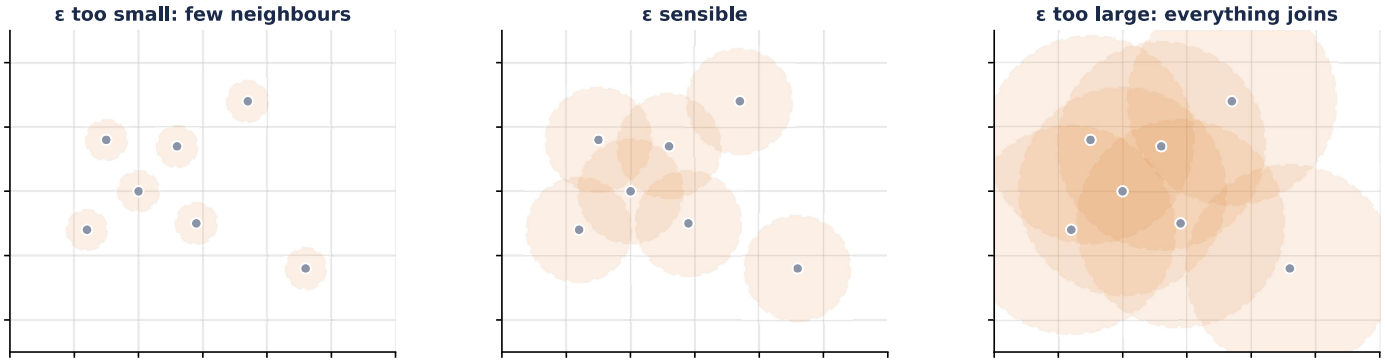
Essential Vocabulary (2/2): ε and MinPts

ε (epsilon): the radius

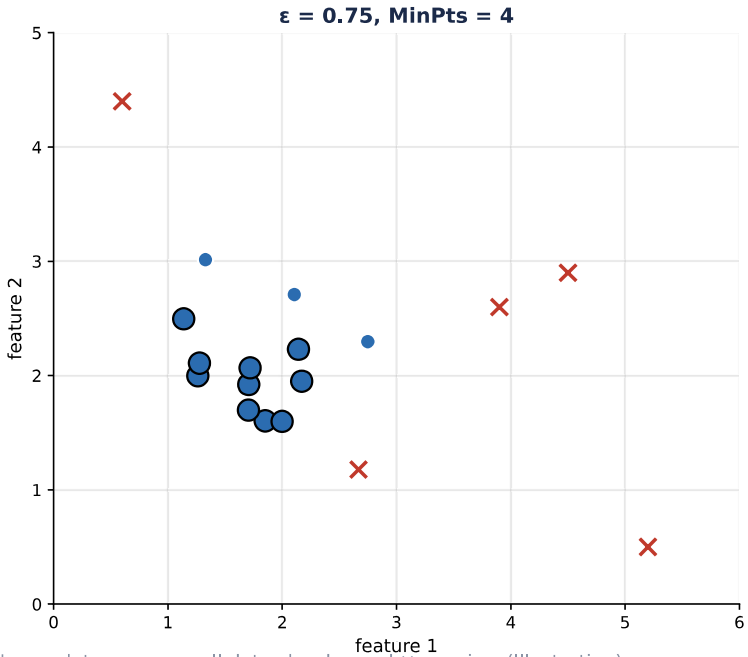
How far we look around a point. Same units as the features (so scaling matters).

MinPts: the crowd size

The minimum number of points (counting the point itself) in its ε-neighbourhood for the area to count as dense.



Three Kinds of Points



Large dot = core, small dot = border, red × = noise. (Illustration)

Core point

Has at least MinPts points (itself included) within ε. It sits inside a dense region.

Border point

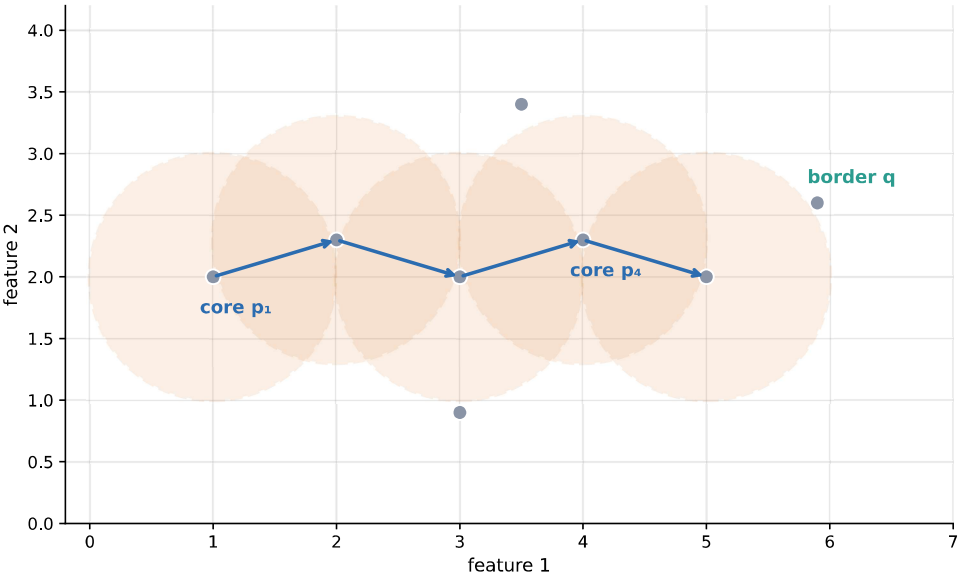
Not core itself, but lies within ε of a core point. It is on the edge of a cluster.

Noise point

Neither core nor within ε of a core point. It belongs to no cluster.

How Clusters Grow: Reachability

A cluster is built by hopping from core point to core point, each within ε of the last.



Directly density-reachable

q lies within ε of a core point p.

Density-reachable

q can be reached from p by a chain of such direct hops.

Density-connected

p and q are both reachable from some core point o.

Mathematics (1/2): Neighbourhood and Core

ε-neighbourhood

$$N_\varepsilon(p) = \{\, q \in D :\ d(p,q) \le \varepsilon \,\}$$

All points of the dataset D that are within distance ε of p. It always contains p itself, since d(p,p) = 0.

Core point condition

$$p \text{ is core} \Leftrightarrow |N_\varepsilon(p)| \ge \text{MinPts}$$

The neighbourhood must contain at least MinPts points, counting p itself.

D	the dataset of n observations
p, q	individual points (observations)
$d(p, q)$	Euclidean distance between p and q
ε	neighbourhood radius (parameter)
MinPts	minimum number of points for a dense area (parameter)
$\|N\|$	number of points in the set N

Mathematics (2/2): Reachability and Clusters

Directly density-reachable

$q \in N_\varepsilon(p)$ and p is core

q is a neighbour of core point p. Not symmetric: a border point cannot reach a core point this way.

Density-reachable

$p = p_1 \to p_2 \to \cdots \to p_m = q$

Each p_{i+1} is directly reachable from p_i. All p_i before q must be core.

Density-connected

$\exists\, o:\ o \Rightarrow p$ and $o \Rightarrow q$

Both p and q are reachable from one core point o. This relation is symmetric.

A cluster C is a non-empty set of density-connected points that is maximal:
if p is in C and q is density-reachable from p, then q is in C too. Noise = every point that belongs to no cluster.

The Algorithm (DBSCAN)


```
Input : data D, eps, MinPts
Output: label for each point (-1 = noise)

1  mark all points UNVISITED; C <- 0
2  for each unvisited point p:
3     N <- points within eps of p
4     if |N| < MinPts:
5        label p NOISE (may change later)
6     else:                        # p is core
7        C <- C+1;  label p with C
8        Q <- N minus {p}          # queue
9        while Q is not empty:
10          q <- next point in Q
11          if q is unvisited or NOISE:
12             label q with C
13          if |N(q)| >= MinPts:
14             add unseen points of N(q) to Q
15 return labels
```

1 Initialise

All points unvisited; no clusters yet

2 Test a point

Count its ε-neighbours

3 Noise or core

Fewer than MinPts: tentative noise. Else: start a cluster

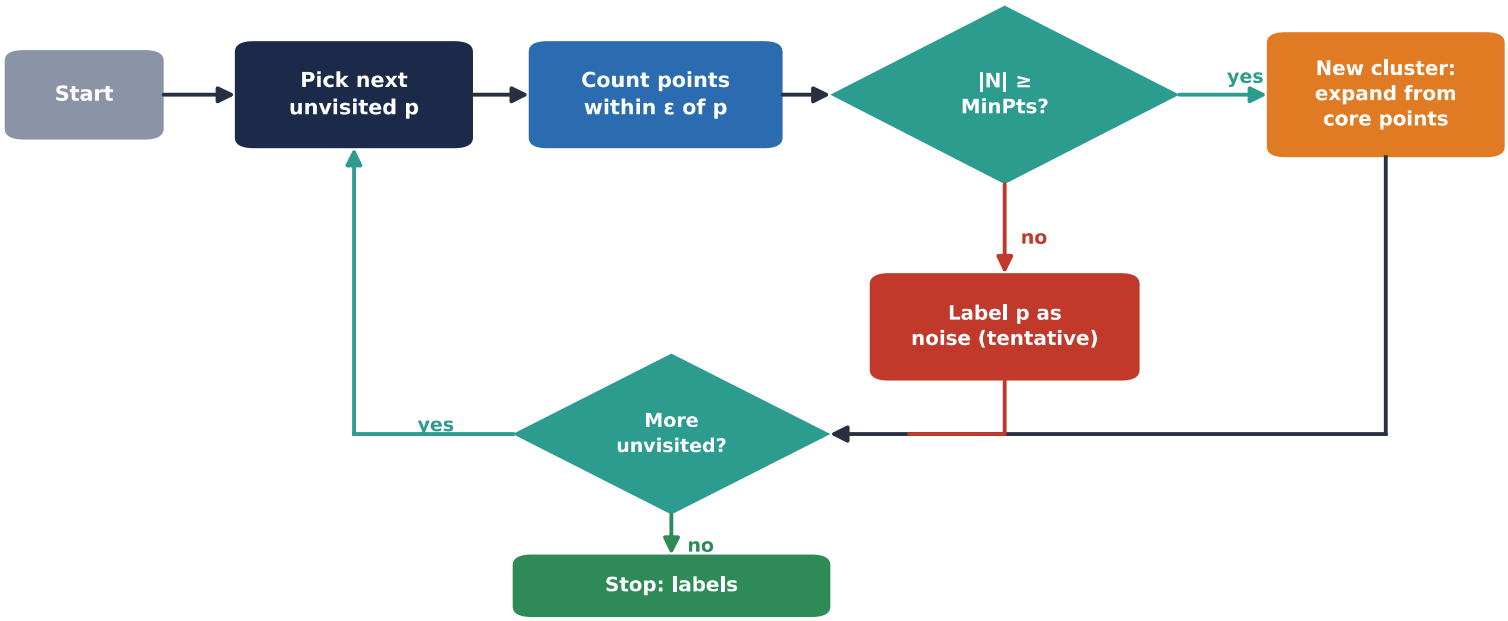
4 Expand

Visit neighbours; core ones add their own neighbours

5 Terminate

When no unvisited points remain

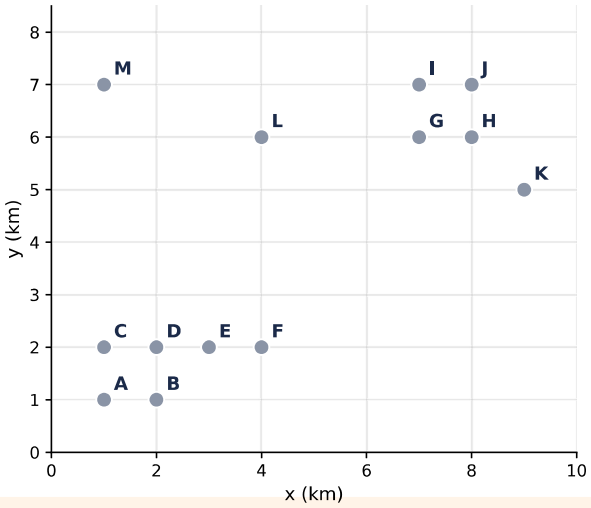
Flowchart



Worked Example: Delivery Locations

13 delivery requests on a city grid (x = east, y = north, in km)

Pt	x	y	Pt	x	y
A	1	1	H	8	6
B	2	1	I	7	7
C	1	2	J	8	7
D	2	2	K	9	5
E	3	2	L	4	6
F	4	2	M	1	7
G	7	6			



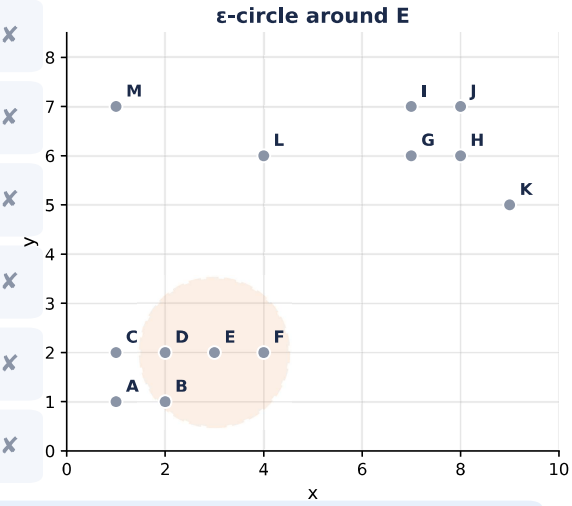
Settings used in this example (specific to this example)

• ε = 1.5 km and MinPts = 3 (the point itself counts). • Distance: Euclidean. • Points are examined in the order A, B, C, …, M.
• Neighbours are processed first-in, first-out. A border point joins the first cluster that reaches it.
• Goal: find dense groups of requests and flag isolated requests. K is not given.

Step 1: Distances from One Point

Take point E = (3, 2). Which points are within ε = 1.5? Distance to every other point:

A: $\sqrt{(1-3)^2+(1-2)^2} = 2.24$ ✘	H: $\sqrt{(8-3)^2+(6-2)^2} = 6.40$ ✘
B: $\sqrt{(2-3)^2+(1-2)^2} = 1.41$ ✔	I: $\sqrt{(7-3)^2+(7-2)^2} = 6.40$ ✘
C: $\sqrt{(1-3)^2+(2-2)^2} = 2.00$ ✘	J: $\sqrt{(8-3)^2+(7-2)^2} = 7.07$ ✘
D: $\sqrt{(2-3)^2+(2-2)^2} = 1.00$ ✔	K: $\sqrt{(9-3)^2+(5-2)^2} = 6.71$ ✘
F: $\sqrt{(4-3)^2+(2-2)^2} = 1.00$ ✔	L: $\sqrt{(4-3)^2+(6-2)^2} = 4.12$ ✘
G: $\sqrt{(7-3)^2+(6-2)^2} = 5.66$ ✘	M: $\sqrt{(1-3)^2+(7-2)^2} = 5.39$ ✘



N(E) = {B, D, E, F} → |N(E)| = 4 ≥ MinPts = 3

E is a CORE point. (Check: ✔ means distance ≤ 1.5, so the point is inside the circle.)

◄ Previous

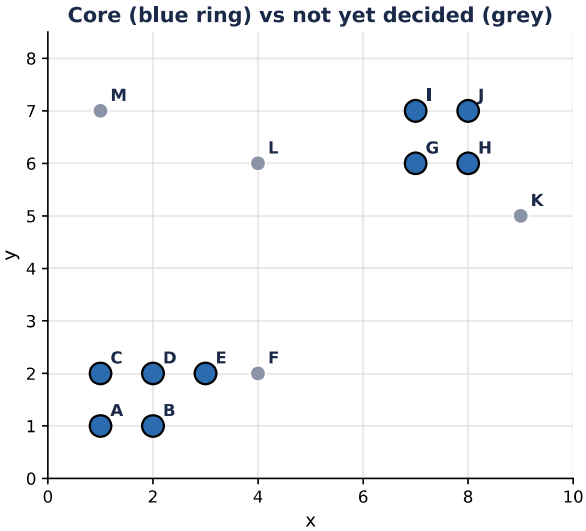
Slide 17 of 33

Next ►

Step 2: Find the Core Points

Repeat the neighbour count for every point (distances computed the same way).

Pt	N(p), incl. itself	|N|	Core?
A	{A, B, C, D}	4	YES
B	{A, B, C, D, E}	5	YES
C	{A, B, C, D}	4	YES
D	{A, B, C, D, E}	5	YES
E	{B, D, E, F}	4	YES
F	{E, F}	2	no
G	{G, H, I, J}	4	YES
H	{G, H, I, J, K}	5	YES
I	{G, H, I, J}	4	YES
J	{G, H, I, J}	4	YES
K	{H, K}	2	no
L	{L}	1	no
M	{M}	1	no



Core points: {A, B, C, D, E, G, H, I, J} (9 of 13)

Non-core: {F, K, L, M} → border or noise? (next step)

◄ Previous

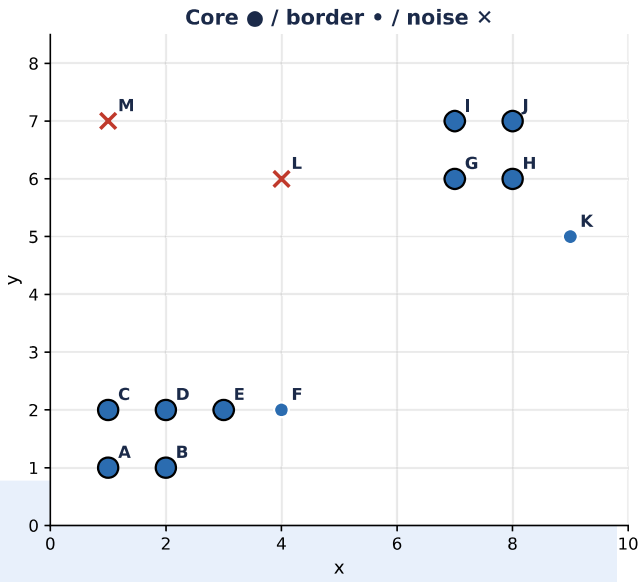
Slide 18 of 33

Next ►

Step 3: Border Point or Noise?

For each non-core point: is there a core point within ε?

Pt	Nearest core point	Distance	≤ 1.5?	Type
F	E (3, 2)	1.00	yes	BORDER
K	H (8, 6)	1.41	yes	BORDER
L	G (7, 6)	3.00	no	NOISE
M	C (1, 2)	5.00	no	NOISE



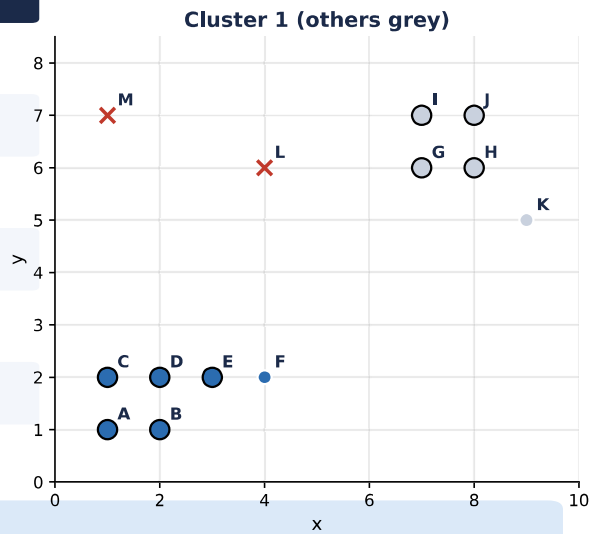
Result: border = {F, K} noise = {L, M}

Noise points are farther than ε from every core point, so they get no cluster.

Step 4: Grow Cluster 1

Start from the first unvisited core point and grow Cluster 1 through its core points.

#	Take point	Type	Adds to queue	Queue afterwards
1	A	core: start	{B, C, D}	[B, C, D]
2	B	core	{E}	[C, D, E]
3	C	core	none	[D, E]
4	D	core	none	[E]
5	E	core	{F}	[F]
6	F	border	none	empty



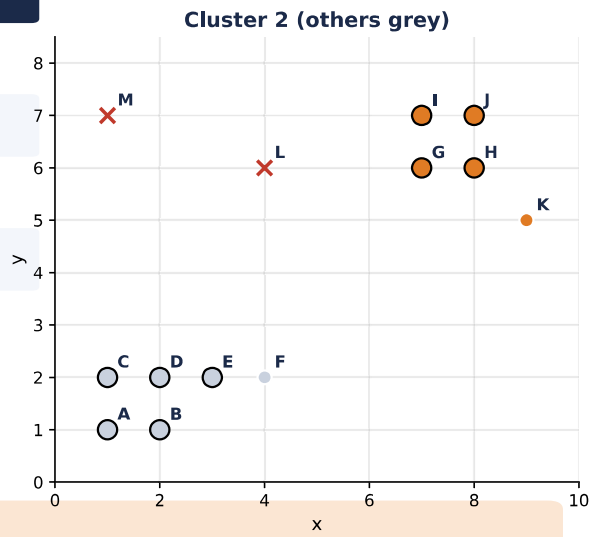
Queue empty → Cluster 1 = {A, B, C, D, E, F} (5 core, 1 border)

Border point(s) F were added but did not add neighbours of their own.

Step 5: Grow Cluster 2

Start from the first unvisited core point and grow Cluster 2 through its core points.

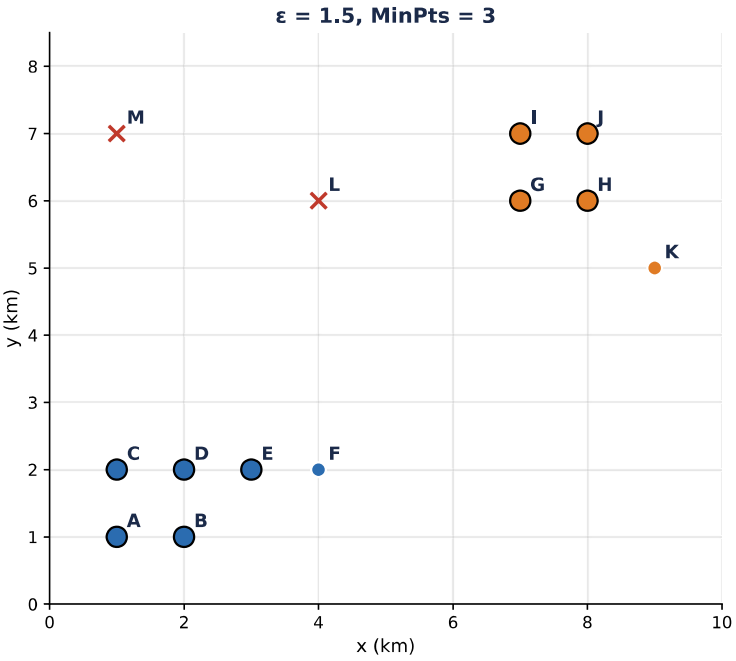
#	Take point	Type	Adds to queue	Queue afterwards
1	G	core: start	{H, I, J}	[H, I, J]
2	H	core	{K}	[I, J, K]
3	I	core	none	[J, K]
4	J	core	none	[K]
5	K	border	none	empty



Queue empty → Cluster 2 = {G, H, I, J, K} (4 core, 1 border)

Border point(s) K were added but did not add neighbours of their own.

Interpreting the Final Result



large dot = core, small dot = border, red × = noise

Group	Members	Core	Border
Cluster 1	{A, B, C, D, E, F}	{A, B, C, D, E}	{F}
Cluster 2	{G, H, I, J, K}	{G, H, I, J}	{K}
Noise	{L, M}	–	–

Reading the clusters

Two dense groups of requests: a south-west hub and a north-east hub.

Reading the noise

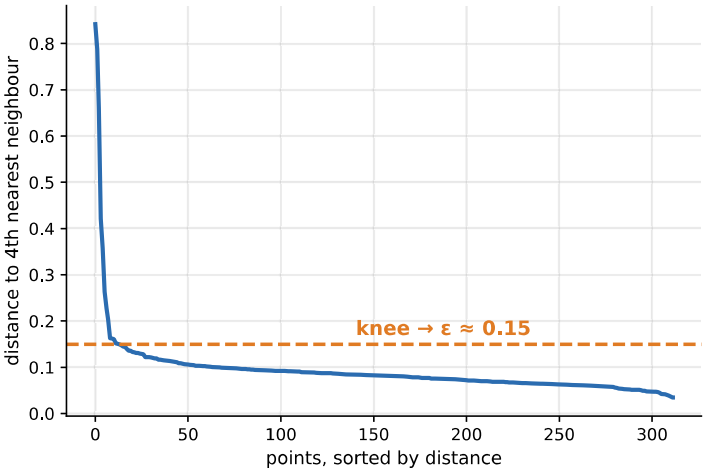
L and M are isolated requests: serve separately or check for errors.

Contrast: K-means with K = 2 gives every point a cluster, forcing L and M into one: L→2, M→1. A different ε or MinPts would change our result.

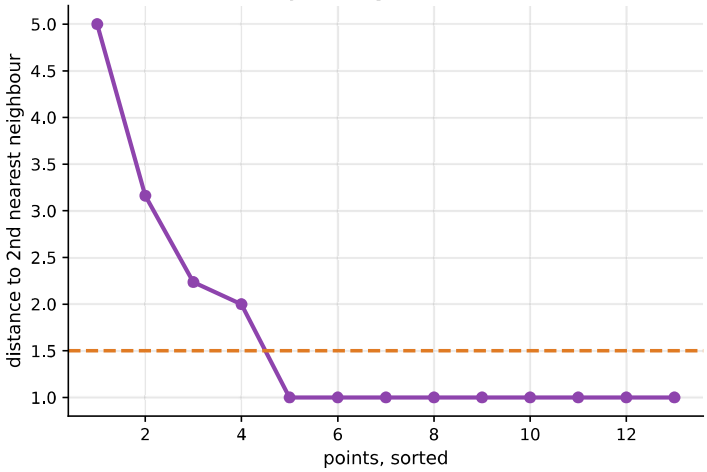
Choosing ε: The k-distance Plot

For each point compute the distance to its (MinPts − 1)-th nearest other point, sort from largest to smallest, and plot. The knee separates dense points (small values) from noise (large values).

Moons data (MinPts = 5)



Delivery example (MinPts = 3)



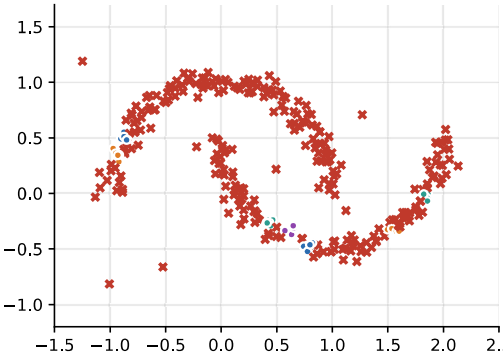
Rule: set ε near the knee. Points to the left of the knee are mostly noise. This is the same neighbour-distance idea as in KNN.

The four largest values belong to the non-core points M, L, K, F. Our ε = 1.5 sits just above the core points' values (1 to 1.41).

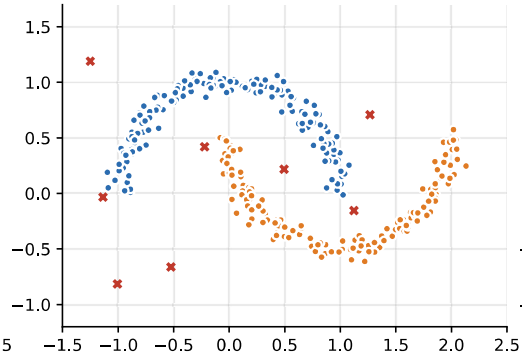
Effect of ε and MinPts

Same data, three choices of ε (MinPts = 5): the result is sensitive.

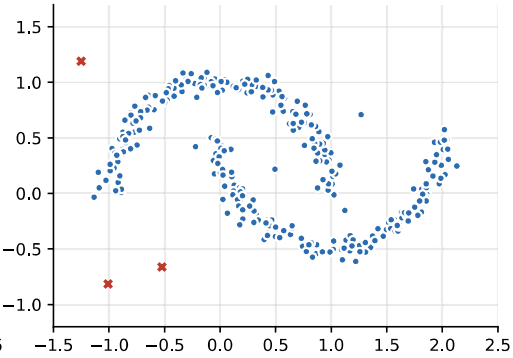
ε = 0.05: 7 cluster(s), 270 noise



ε = 0.15: 2 cluster(s), 8 noise



ε = 0.6: 1 cluster(s), 3 noise



Too small: fragments and lots of noise. **About right: two moons + outliers flagged.** **Too large: clusters merge into one.**

MinPts rule of thumb: MinPts ≥ p + 1 (p = number of features); often 2p. Larger MinPts = stricter density, more noise.

Practical Considerations (1/2)

Scale the features

ε is one radius for all features, so a feature in large units dominates. Standardise first, as in K-means.

Distance metric

Euclidean is default; Manhattan, cosine or geographic (haversine) distance suit other data. ε must match the metric.

High dimensions

Distances become similar in many dimensions, so ε loses meaning. Reduce dimensions first (e.g. PCA).

Border-point ambiguity

A border point near two clusters joins the first one that reaches it, so the order of the data can change its label.

Missing values

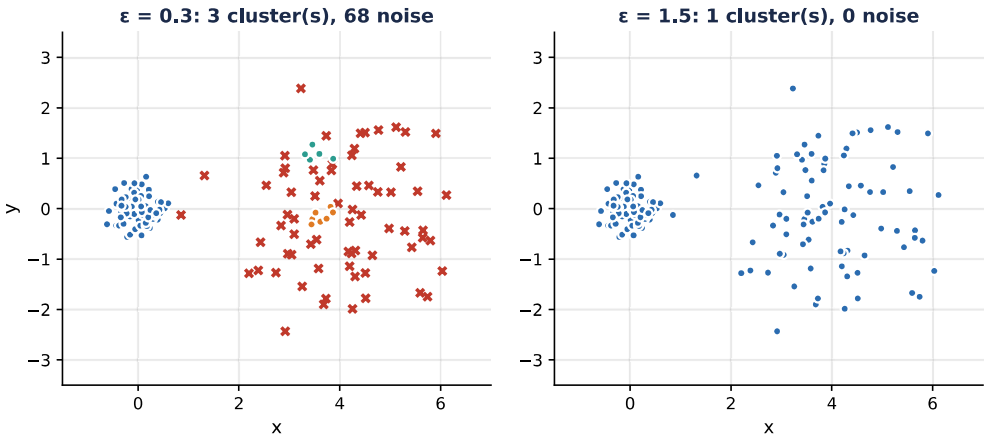
Impute before clustering; distances cannot use NaN.

Large data

With a spatial index (k-d tree) neighbour search is fast; without it the cost grows like n^2.

Practical Considerations (2/2): Varying Density

One global ε cannot suit both a tight cluster and a loose cluster.



What to do

Small ε finds the dense cluster but calls the sparse one noise. Large ε captures both but may merge or absorb noise.

Try OPTICS or HDBSCAN, which handle varying density.

This is DBSCAN's main limitation, and it is why density extensions exist.

DBSCAN vs K-means vs Hierarchical vs HDBSCAN

	DBSCAN	K-means	Hierarchical	HDBSCAN
Needs K?	No	Yes	No	No
Shape	Arbitrary	Round	Depends	Arbitrary
Varying density	Poor	n/a	Moderate	Good
Noise points	Yes	No	No	Yes
Main parameters	ε, MinPts	K	Linkage, cut	Min cluster size
Use when	Dense, irregular, outliers	Round, large, K known	Want a tree / small data	Densities differ

Next in the course: hierarchical clustering and dendrograms.

Quick Quiz

Choose an answer for each question, then turn the page.

1. A point has at least MinPts points within ε (itself included). It is a…

(a) Border point (b) Core point (c) Noise point (d) Centroid

2. Which of these does DBSCAN not require you to specify?

(a) ε (b) MinPts (c) Number of clusters K (d) A distance measure

3. Which points can add new points to a growing cluster?

(a) Core points only (b) Border points only (c) Noise points (d) All points

4. ε is far too large. The most likely outcome is…

(a) Many tiny clusters (b) All noise (c) One giant cluster (d) Exactly K clusters

5. Why is DBSCAN a poor choice for clusters of very different densities?

(a) It needs K (b) One global ε fits only one density (c) It ignores distance (d) It needs labels

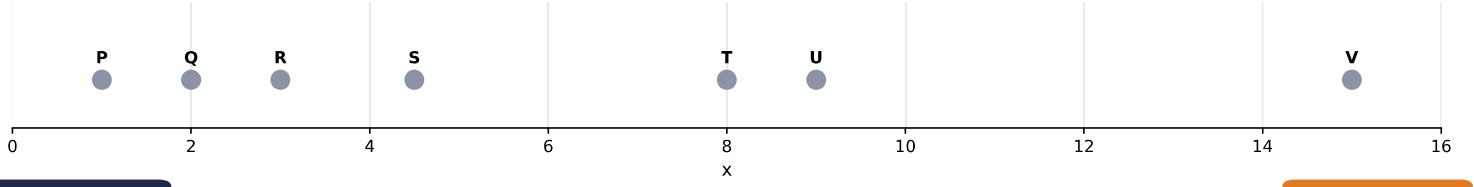
Exercise: Do It by Hand

One feature, seven points: ε = 1.5, MinPts = 3.

Pt	P	Q	R	S	T	U	V
x	1	2	3	4.5	8	9	15

- List the ε-neighbourhood of every point (distance ≤ 1.5) and its size.
- Which points are core?
- Classify the remaining points as border or noise.
- How many clusters are there, and which points form them?

Try it before turning the page.



Exercise: Solution

Pt	N(p)	|N|	Type
P	{P, Q}	2	BORDER
Q	{P, Q, R}	3	CORE
R	{Q, R, S}	3	CORE
S	{R, S}	2	BORDER
T	{T, U}	2	NOISE
U	{T, U}	2	NOISE
V	{V}	1	NOISE

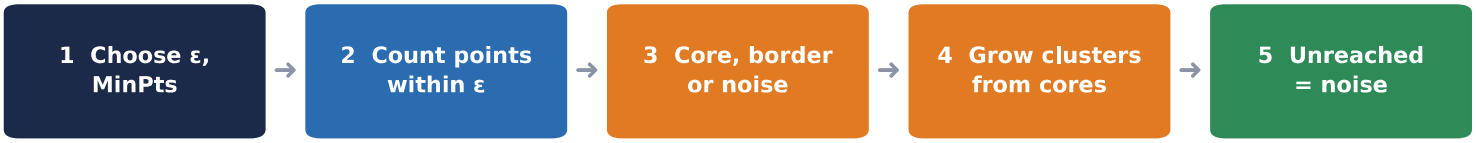
Reasoning

Q and R have 3 points in their neighbourhoods, so they are core. P and S have only 2, but each lies within 1.5 of a core point (P–Q = 1, S–R = 1.5), so they are border points. T, U, V have no core point within 1.5: noise.

Result: 1 cluster {P, Q, R, S}
Noise: {T, U, V}

Note: S–R distance is exactly 1.5, and the rule is ≤ ε, so S is inside the neighbourhood.

Summary and Take-Home Message



Key learning points

- DBSCAN defines clusters as dense regions separated by sparse space, so shapes can be arbitrary and K is not needed.
- Core points (≥ MinPts within ε) form the backbone; border points hang on the edge; everything else is noise.
- Our example: 13 points, ε = 1.5, MinPts = 3 gave 2 clusters and 2 noise points (L, M).
- Results depend on ε, MinPts, scaling and density differences: use the k-distance plot and domain knowledge.
- Next in the course: hierarchical clustering and dendrograms.

Take-home: ask 'is this region crowded enough?' instead of 'how far is this point from a centre?'